

# 챕터 1. 왜 컨테이너인가

입사 3개월 차 오픈이는 기능 하나를 손보고 있었습니다. 로컬 PC에서는 테스트가 모두 통과했고, 남은 건 운영 서버에 올리는 일뿐이었습니다.

빌드한 파일을 서버에 올리고 실행 명령을 입력했습니다. 그런데 터미널이 한 박자 멈춘 뒤 빨간 글씨가 화면을 채웠습니다. 자바 버전이 맞지 않는다는 오류였습니다.

자바 버전을 서버에 맞추자 이번엔 다른 오류가 떴습니다. 라이브러리가 의존하는 시스템 파일이 서버에 없었습니다. 그 파일을 설치하니 곧바로 설정 파일의 경로가 서버의 디렉터리 구조와 어긋났습니다.

'내 PC에선 분명히 됐는데.'

하나를 해결하면 곧바로 또 다른 오류가 터졌습니다. 코드는 그대로인데 서버의 환경만 조금씩 어긋나 있었습니다.

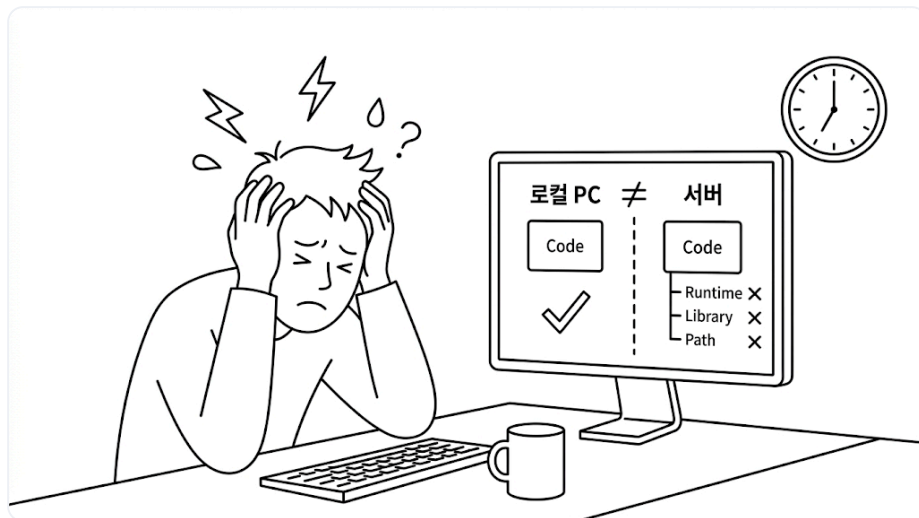


그림 1-1. 환경 불일치로 막힌 오픈이

한참을 헤매고 있을 때, 옆자리 선배가 모니터를 잠깐 들여다봤습니다.

**선배** : "환경이 달라서 그래요. **도커(Docker)** 한번 알아봐요."

선배의 말을 들은 오픈이는 퇴근 후 유튜브에 **도커**를 검색했습니다. 그중 가장 눈에 띈 영상은 도커와 컨테이너의 유래를 다룬 내용이었습니다.

## 1.1 컨테이너 - 항구에서 빌려온 이름

### 1.1.1 하역에 며칠씩 걸리던 시절

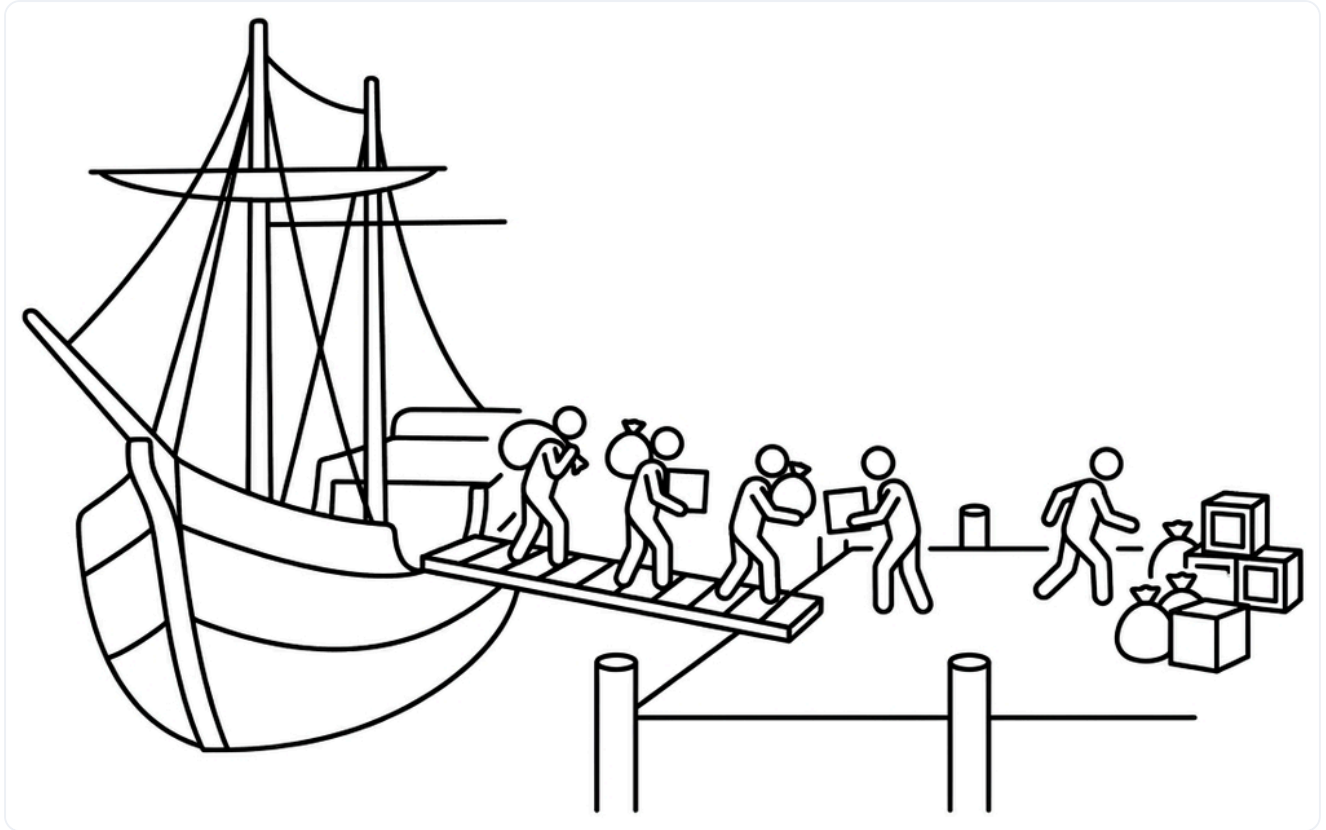


그림 1-2. 컨테이너가 없던 시절의 해상 물류

1950년대 미국 동부의 한 항구. 수십 명의 인부가 화물을 일일이 어깨에 메고 부두로 옮기고 있었습니다. 자루에 담긴 곡물, 묶음으로 된 나무 상자, 무거운 드럼통, 제각각인 길이의 철근 다발까지 화물의 모양은 천차만별이었습니다.

배를 다 비우는 데 며칠이 걸리던 시절이었습니다. 규격이 제각각이라 분류부터 트럭 적재까지 모두 사람 손을 거쳤습니다. 그 때문에 화물을 실어 가려는 트럭 운전사들은 줄 끝에서 며칠씩 차례를 기다렸습니다.

그 줄 한가운데에 트럭 운전사 **말콤 맥린**이 있었습니다. 지루한 기다림 속에서 그는 한 가지 의문을 품었습니다. '**왜 화물을 통째로 옮기지 않을까?**'

### 1.1.2 규격을 통일하다

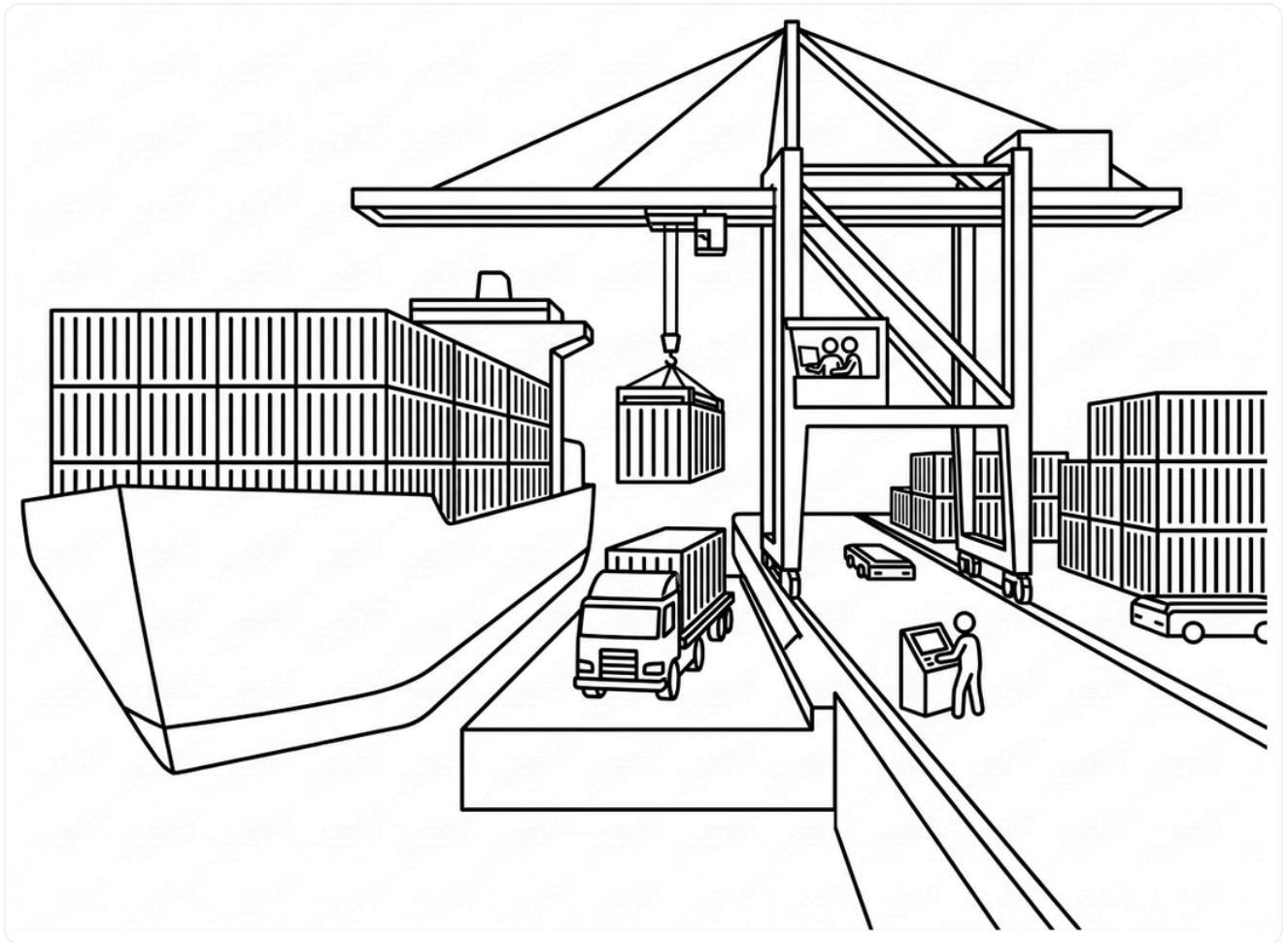


그림 1-3. 표준 컨테이너와 크레인으로 바뀐 현대 항구

맥린이 찾은 답은 **표준 규격의 상자**였습니다. 화물을 낱개로 다루지 말고, 크기와 모양이 똑같은 상자에 담아 통째로 옮기자는 것이었습니다.

규격이 생기자 항구가 달라졌습니다. 크레인이 상자를 통째로 들어 배에 싣고, 도착지에서도 같은 방식으로 내렸습니다. 안에 무엇이 들었든 **규격만 같으면 처리 과정은 동일**했습니다.

덕분에 며칠씩 걸리던 하역이 몇 시간으로 줄었습니다. 맥린은 배와 트럭을 잇는 운송 시스템까지 구축했고, 그렇게 '담는 그릇'이라는 뜻의 **컨테이너(Container)**는 비로소 **세계 물류의 표준**이 되었습니다.

### 1.1.3 IT로 건너온 컨테이너

**컴퓨터마다 다른 환경**은 소프트웨어를 옮길 때 늘 걸림돌이 됩니다. 운영체제나 라이브러리, 설정이 조금만 달라도 오류가 발생할 수 있기 때문입니다.

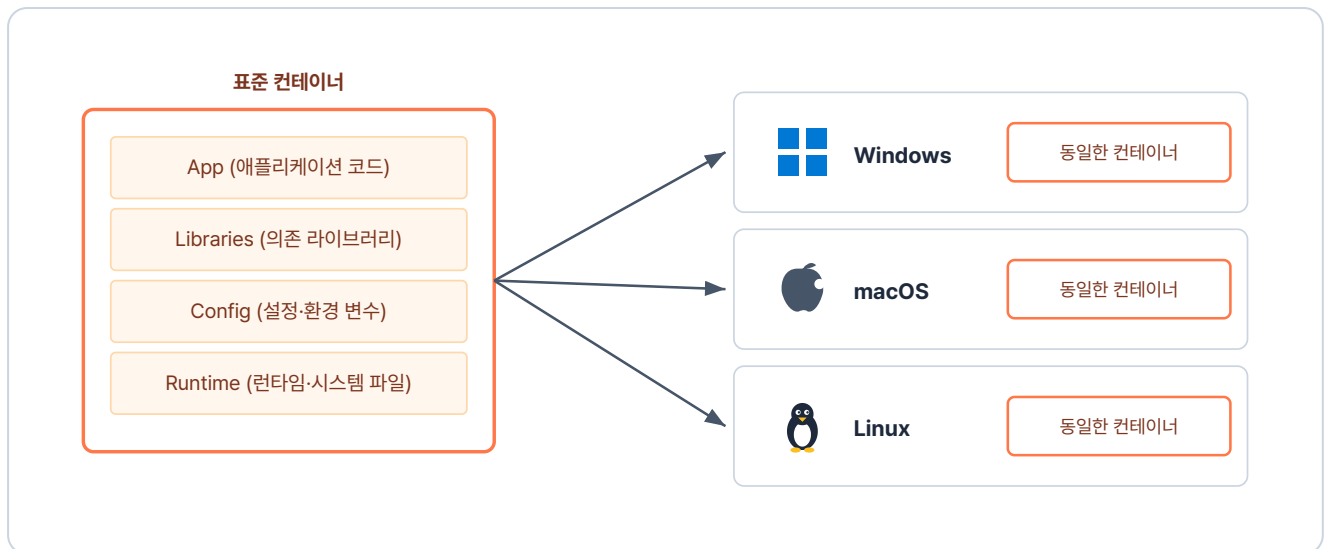


그림 1-4. 소프트웨어도 똑같이 표준 컨테이너에 담아 어디서든 같은 결과로 실행

물류가 컨테이너로 표준을 세웠듯, 소프트웨어도 **컨테이너**라는 개념으로 이 문제를 해결했습니다. 실행에 필요한 파일과 설정을 독립된 컨테이너에 담아 통째로 옮깁니다. 그러면 인프라 환경이 어떻게 바뀌든, 만든 프로그램이 늘 **같은 환경에서 안정적으로** 돌아갑니다.

**컨테이너(Container):** 애플리케이션과 그 애플리케이션이 필요로 하는 모든 것(라이브러리, 설정, 시스템 파일)을 하나의 상자에 담아, 어디서 실행해도 같은 결과를 내는 실행 단위입니다.

## 1.2 Docker와 Kubernetes - 공장과 관제 센터

컨테이너를 쓰려면 먼저 그것을 만들어낼 도구가 필요합니다. IT에서 그 역할을 하는 도구가 **Docker**입니다.

Docker는 **이미지**라는 설계도를 바탕으로 작동합니다. 이미지에 실행 환경을 미리 정의해 두면, 똑같은 환경의 컨테이너를 언제 어디서든 몇 개든 만들어낼 수 있습니다.

문제는 컨테이너가 수백, 수천 개로 늘어날 때입니다. 어느 서버에 올릴지, 문제가 생긴 컨테이너는 어떻게 교체할지 사람이 일일이 관리하기란 불가능에 가깝습니다. 항구에서 수많은 컨테이너를 조율하는 **관제 시스템**이 필요한 것과 같습니다.

소프트웨어에서 이 관제 시스템 역할을 하는 것이 **Kubernetes**입니다. "이 서비스를 항상 3개 유지해줘"처럼 **원하는 상태**를 선언해 두면, Kubernetes가 컨테이너 상태를 계속 지켜봅니다. 하나가 죽으면 자동으로 새 컨테이너를 띄우고, 필요에 따라 수를 늘리거나 줄이며 전체를 조율합니다.

'Docker가 컨테이너를 만들고, Kubernetes가 그걸 관리하는구나.'

Docker와 Kubernetes의 역할을 정리하면 다음과 같습니다.

| 구분    | Docker          | Kubernetes           |
|-------|-----------------|----------------------|
| 역할    | 컨테이너 만들고 실행     | 컨테이너 운영 자동화          |
| 비유    | 표준 컨테이너를 찍는 공장  | 수많은 배를 관리하는 항만 관제 센터 |
| 핵심 기능 | 이미지 빌드, 컨테이너 실행 | 자동 복구, 스케일링, 무중단 배포  |

## 1.3 학습 지도 - Docker에서 Kubernetes까지

이 책의 학습 여정은 다섯 챕터로 이어집니다.



그림 1-5. 책 한 권 분량의 학습 여정. 한 단계씩 다음 챕터로 나아갑니다

각 챕터에서 다루는 내용은 다음과 같습니다.

- 챕터 2에서는 Docker로 컨테이너가 격리되는 원리를 익히고, 직접 실행해 이미지로 만듭니다.
- 챕터 3에서는 Dockerfile과 Docker Compose로 여러 컨테이너를 한 번에 다룹니다.
- 챕터 4에서는 Kubernetes의 Pod와 Deployment로 컨테이너를 자동 복구·확장합니다.

- 챕터 5에서는 Service와 Ingress로 매번 바뀌는 주소를 고정하고 외부에서 접속할 수 있게 합니다.
- 챕터 6에서는 Secret과 Volume으로 설정·데이터를 관리합니다.

이 책은 코드 작성보다 컨테이너의 전체 개념과 흐름을 이해하고, 단계별로 실습하며 익히는 것을 목표로 합니다. 한 단계씩 학습하며 Docker와 Kubernetes의 큰 흐름을 따라가 보겠습니다.

### 이것만은 기억하자

- **컨테이너는 표준 상자입니다.** 말콤 맥린이 화물을 표준 컨테이너에 담아 어디든 같은 방식으로 옮긴 것처럼, IT 컨테이너도 애플리케이션과 그에 필요한 모든 것을 한 상자에 담아 어디서든 같은 결과를 냅니다.
- **Docker는 공장, 이미지는 설계도, 컨테이너는 찍혀 나온 결과물입니다.** 이미지 하나로 같은 모양의 컨테이너를 여러 개 찍어낼 수 있습니다.
- **Kubernetes는 항만 관제 센터입니다.** 여러 컨테이너를 자동으로 관리합니다. 사람이 "원하는 상태"만 선언하면 시스템이 그 상태를 유지합니다.